

Técnicas de control inteligente aplicadas al robot educativo Lego Mindstorms EV3

Javier Arambarri Calvo,
Alumno del Máster en Ingeniería de Sistemas y Control - UCM y UNED,
jaaramba@ucm.es, javierarambarricalvo@gmail.com,
Asignatura: Control Inteligente, Curso académico 2025-2026,
Profesora: Matilde Santos Peñas.

Resumen

Este trabajo presenta el diseño e implementación de diferentes técnicas de control inteligente aplicadas a un robot diferencial educativo Lego Mindstorms EV3. Se desarrollan controladores borrosos, proporcionales optimizados mediante algoritmos genéticos y controladores neuronales, con el objetivo de corregir desviaciones en la trayectoria sin necesidad de depender exclusivamente de sensores externos o de la sintonización manual de controladores clásicos. Los resultados validan la viabilidad de estas técnicas en un entorno educativo de robótica.

Palabras clave: robótica educativa, control inteligente, controlador borroso, controlador neuronal, optimización, simulación.

1 Introducción

Los motores Lego Mindstorms EV3 [11] pueden pertenecer a distintas generaciones, lo que implica diferencias en par y velocidad. Lego recomienda utilizar motores emparejados de la misma generación para minimizar estas discrepancias. Sin embargo, este proyecto busca ofrecer una solución de control inteligente que permita prescindir de esa restricción, garantizando un movimiento coordinado incluso con motores heterogéneos o sometidos a condiciones adversas.

Cuando se ordena a los motores del robot diferencial avanzar a la misma velocidad nominal, la velocidad real puede diferir debido a variaciones de fabricación, desgaste o fricción. Esto provoca que el robot no mantenga una trayectoria recta, figura 1. Además, factores externos como un cable rozando con un motor pueden reducir drásticamente su velocidad, generando una desviación aún mayor. Una solución ingenua sería reducir la velocidad del motor no afectado hasta igualarla con la del motor frenado, pero esta estrategia no es suficiente debido a la relación no lineal y desconocida entre señal de control y velocidad real.

El objetivo de este trabajo es diseñar e implementar diferentes técnicas de control inteligente como borroso, proporcional optimizado mediante algoritmos genéticos y neuronal, que permitan corregir desviaciones de trayectoria. El proyecto se plantea como una puesta en práctica de las técnicas estudiadas en la asignatura utilizando un kit educativo de robótica.

La implementación del proyecto se encuentra en la rama “ci-misc” del repositorio <https://github.com/arambarricalvoj/eduROS2/tree/ci-misc>.

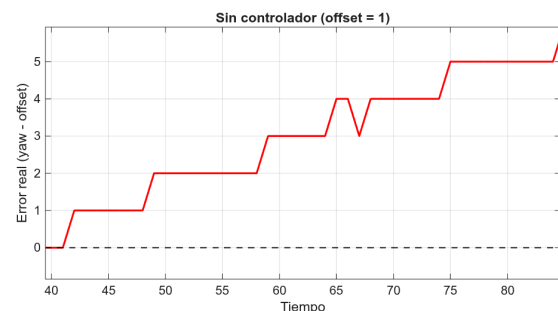


Figura 1: Señal de error sin controlador.

2 Metodología

Dado el carácter académico del trabajo, contemplado como proyecto final de una asignatura de 6 créditos ECTS, la metodología seguida únicamente contempla experimentación directa con el robot Lego Mindstorms EV3, y la combinación con simulación en entornos de *software*, que permitiría obtener mejores y más completos resultados, se propone como línea futura de trabajo.

En este sentido, el sistema se entrenó y probó en condiciones controladas y con perturbaciones introducidas manualmente para evaluar la robustez de los controladores.

Estas perturbaciones al no ser sistemáticas ni homogéneas introducen sesgo, pero son suficientes para cumplir con el objetivo del proyecto y su introducción sistemática y controlada mediante simulación se plantea también como línea futura.

Para calcular la desviación de trayectoria sin

ayuda de sensor de giro, se emplea el modelo geométrico del robot.

Se ha utilizado *ROS2 Jazzy Jalisco* [13] como marco de integración, junto con programación en *Python* y *C++*. Para posibilitar la comunicación con el hardware se empleó el puente *eduROS2* [2].

3 Modelo geométrico del robot diferencial

El modelo geométrico del robot diferencial, figura 2, se basa en la relación entre las velocidades de sus dos ruedas y la trayectoria resultante, y ha sido ampliamente estudiado en la literatura [18]. Cada rueda, de diámetro D , genera una velocidad lineal proporcional a su velocidad angular, mientras que la diferencia entre ambas determina la velocidad angular del chasis (*yaw rate*). El parámetro fundamental del modelo es la distancia entre ruedas, denominada *axle track* L , que actúa como factor de escala en el cálculo de la rotación.

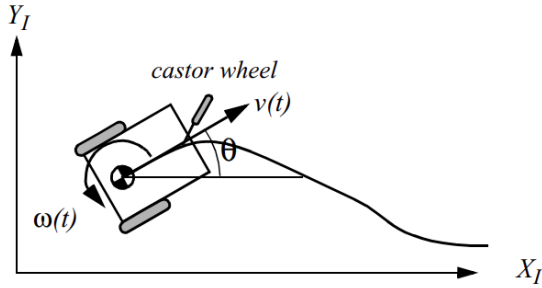


Figura 2: Cinemática diferencial. Fuente: [18].

La orientación del robot puede actualizarse de dos formas complementarias:

3.1 A partir de distancias recorridas

Si d_{left} y d_{right} representan las distancias lineales recorridas por las ruedas izquierda y derecha, respectivamente, el incremento angular del robot en un intervalo de tiempo se calcula como:

$$\Delta\theta = \frac{d_{right} - d_{left}}{L} \quad (1)$$

donde $\Delta\theta$ corresponde al cambio de orientación entre dos lecturas de *encoders*. La orientación acumulada se obtiene sumando sucesivamente estos incrementos:

$$\theta(t) = \sum \Delta\theta \quad (2)$$

3.2 A partir de velocidades instantáneas

Si se consideran las velocidades lineales de cada rueda v_{left} y v_{right} , la velocidad angular instantánea del robot se expresa como:

$$\dot{\theta} = \frac{v_{right} - v_{left}}{L} \quad (3)$$

donde $\dot{\theta}$ es la tasa de giro del chasis (*yaw rate*). En este caso, la orientación se obtiene integrando la velocidad angular en el tiempo:

$$\theta(t) = \int \dot{\theta}(t) dt \quad (4)$$

De esta forma, el modelo permite trabajar tanto con incrementos discretos de distancia como con velocidades instantáneas, adaptándose a diferentes modos de lectura de los *encoders*.

Una vez descifrada la orientación, se calcula el error en la trayectoria mediante la siguiente operación:

$$error_{\theta(t)} = \theta(t) - \theta(t = 0) \quad (5)$$

donde $\theta(t = 0)$ es la orientación inicial del robot en reposo, previo a comenzar el movimiento. En general, $\theta(t = 0) = 0$, pero en caso de utilizar el sensor de giro de Lego, que proporciona directamente $\theta(t)$, este se podrá establecer a cero o en su defecto, establecer como *offset* el valor actual proporcionado por el sensor. Es decir, si se fija el *offset* $\theta(t = 0) = 57$, por ejemplo, ese valor actuará como el cero.

El robot utilizado, figura 3, tiene un diámetro de rueda $D = 62.4$ mm y una separación entre ruedas $L = 110$ mm.

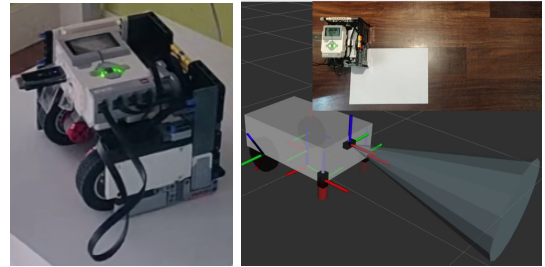


Figura 3: Robot utilizado.

4 Control Fuzzy

El propósito del control borroso es manejar la incertidumbre y la imprecisión en sistemas dinámicos, permitiendo diseñar controladores que no requieren un modelo matemático exacto del proceso. Este tipo de control resulta

especialmente útil cuando la relación entre las variables de entrada y salida es no lineal o difícil de caracterizar, como ocurre en el caso de los motores Lego Mindstorms EV3, cuyos comportamientos pueden variar por desgaste o diferencias de fabricación.

Se basa en la lógica borrosa, introducida por Zadeh [5], que proporciona un marco para representar conocimiento mediante reglas lingüísticas, mientras que Pedrycz [4] desarrolló sus primeras aplicaciones prácticas en sistemas de control. En este proyecto, se emplea un controlador borroso para obtener la corrección que se debe aplicar en la velocidad angular del robot de forma que se mantenga una trayectoria recta, ajustando las consignas en función de las diferencias medidas por los *encoders* y, en una segunda fase, por el sensor de giro.

Respecto a los *encoders*, la desviación de la trayectoria se obtiene de dos maneras diferentes: según la velocidad de cada motor y según la distancias o grados avanzados por cada uno de ellos, siguiendo siempre el modelo geométrico del robot.

Para ajustar el controlador a cada una de las variantes (por velocidad, distancia o sensor de giro), se utilizan ganancias, lo que permite adaptar la salida del controlador sin modificar las funciones de pertenencia, simplificando así la implementación.

4.1 Diseño del controlador borroso

El controlador se concibe como un esquema genérico, aplicable a distintos casos de uso. Su única variable de entrada es el error expresado en porcentaje, lo que permite trabajar indistintamente con errores de velocidad, distancia u orientación. Sin embargo, al emplear el modelo geométrico del robot, el valor que se obtiene corresponde al error de orientación, por lo que en la práctica la implementación recibe siempre esta magnitud como entrada.

De forma coherente, la salida del controlador representa el porcentaje de corrección que debe aplicarse sobre la velocidad angular del robot. Por ejemplo, si el controlador devuelve una salida del 30% y la velocidad lineal deseada es de 25 unidades, la velocidad angular aplicada será el 30% de dicho valor, es decir, 7.5 unidades.

Se definen funciones de pertenencia triangulares superpuestas, figura 4, de modo que cualquier valor dentro del rango de entrada activa al menos un conjunto difuso y produce una salida.

Las reglas lingüísticas definidas para la fase de inferencia se presentan en la tabla 1 y el operador de

agregación empleado es el máximo, utilizado para combinar las reglas activadas de manera intuitiva y consistente, ya que selecciona en cada punto del dominio el mayor grado de pertenencia aportado por las reglas [10].

Finalmente, la *defuzzificación* se realiza mediante el método del centroide [17] con 200 muestras, con el objetivo de conseguir suficiente resolución que asegure transiciones suaves entre las acciones de control.

El resumen de los parámetros principales se presenta en la tabla 2 y los conjuntos borrosos junto con sus rangos en las tablas 3 y 4, donde a y c son los vértices de la base del triángulo.

Tabla 1: Reglas lingüísticas del controlador borroso.

Condición en e_t	Acción en Δv
muy bajo	negativo fuerte
bajo	negativo suave
medio	nulo
alto	positivo suave
muy alto	positivo fuerte

Tabla 2: Parámetros del controlador borroso.

Parámetro	Valor o rango
Entrada: error $\theta(t)$ en %, e_t	[-100, 100]
Salida: corrección vel. angular en %, Δv	[-100, 100]
<i>Defuzzificación</i>	Centroide
Operador de agregación	Máximo
Ganancia velocidad	1.0
Ganancia posición	0.4
Ganancia sensor de giro	2.5

Tabla 3: Funciones de pertenencia definidas para la variable de entrada e_t .

Conjunto	Rango (a,b,c)
muy bajo	(-100, -80, -40)
bajo	(-60, -30, 0)
medio	(-20, 0, 20)
alto	(0, 30, 60)
muy alto	(40, 80, 100)

Tabla 4: Funciones de pertenencia definidas para la variable de salida Δv .

Conjunto	Rango (a,b,c)
negativo fuerte	(-100, -100, -50)
negativo suave	(-60, -30, 0)
nulo	(-20, 0, 20)
positivo suave	(0, 30, 60)
positivo fuerte	(50, 100, 100)

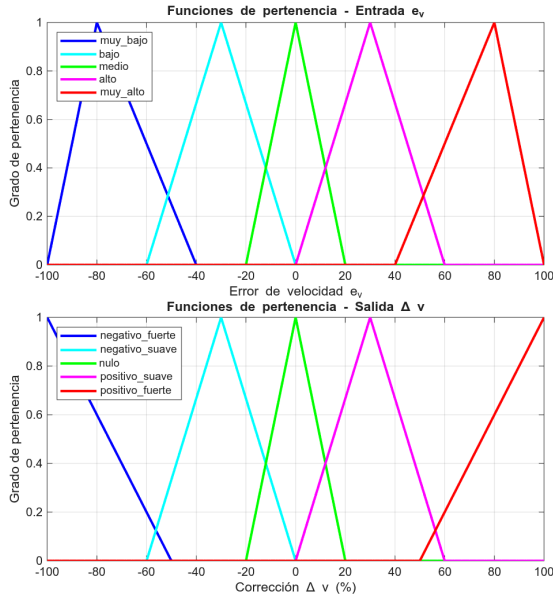


Figura 4: Funciones de pertenencia de entrada y de salida del controlador borroso.

4.2 Implementación

El controlador borroso se implementa en el lenguaje de programación *C++* utilizando la librería *fuzzylite* [16], que permite definir variables de entrada y salida, funciones de pertenencia y reglas lingüísticas.

5 Algoritmos Genéticos

El propósito de los algoritmos genéticos es optimizar parámetros de control mediante técnicas inspiradas en la evolución natural [7], evitando la necesidad de una sintonización manual. En este proyecto se emplea un algoritmo genético (*GA*) para ajustar el parámetro K_p de un controlador proporcional encargado de corregir las desviaciones de trayectoria utilizando el sensor de giro.

La sintonización de este parámetro resulta compleja debido a que no se conoce de manera explícita la relación entre la corrección angular calculada y la velocidad angular aplicada, ya que el robot se controla mediante porcentajes de velocidad en cada rueda. En un escenario ideal, lo óptimo sería llevar a cabo la sintonización completa de un controlador PID, ajustando los parámetros K_p , K_i y K_d . Sin embargo, dado el carácter académico del proyecto y la ausencia de un entorno de simulación, se ha optado por simplificar el diseño utilizando únicamente la acción proporcional, ya que el proceso de prueba y error realizado por el algoritmo genético se ejecuta directamente sobre el robot real.

El proceso evolutivo comienza con una población inicial de valores candidatos de K_p , que se evalúan en función de su desempeño en mantener la trayectoria recta del robot.

A continuación, se aplican los operadores de *selección*, *cruce* y *mutación* para generar nuevas poblaciones, favoreciendo aquellos individuos con mejor rendimiento. De esta forma, el algoritmo converge hacia un valor óptimo de K_p que mejora la estabilidad del controlador proporcional. Del conocimiento de *eduROS2* [2] y la dinámica del robot, se puede acotar el rango de valores posibles de la población, en este caso, $[2.0, 9.0]$, ya que con ese mínimo el robot tarda en corregir y con ese máximo genera demasiadas oscilaciones como se observar en los resultados.

Cada individuo se evalúa por episodios de 4 segundos, cada generación tiene una población de 6 individuos y se ha establecido un máximo de 5 generaciones.

5.1 Función de aptitud

Evalúa el rendimiento de cada individuo y es única para cada aplicación. Se ha diseñado para evaluar las oscilaciones (*osc*) que genera el individuo en estudio y el tiempo de recuperación necesario por el robot. Las oscilaciones se obtienen como la variación media entre salidas consecutivas del controlador, y el tiempo de recuperación (*TR*) como el número de muestras fuera del rango de error permitido, $\pm 2^\circ$.

La función de aptitud diseñada se calcula como:

$$fitness = w_{osc} \cdot osc + w_{TR} \cdot TR \quad (6)$$

donde $w_{osc} = 0.2$ y $w_{TR} = 0.5$ son los pesos asignados arbitrariamente a cada criterio. Los individuos con menor valor de *fitness* son considerados más aptos.

5.2 Selección

Tras evaluar la población inicial mediante la función de aptitud, los individuos se ordenan en función de su aptitud y se conservan los dos de menor valor, ya que son los dos más aptos.

5.3 Cruce

A partir de los individuos seleccionados se generan nuevos candidatos combinando los valores de K_p de los dos progenitores. El cruce implementado es de tipo *interpolación* [9], donde el nuevo individuo (descendiente) se obtiene como una combinación lineal de los progenitores:

$$K_p^{desc.} = \beta \cdot K_p^{progen. 1} + (1 - \beta) \cdot K_p^{progen. 2} \quad (7)$$

con $\beta \in [0,1]$ elegido aleatoriamente en cada cruce. Este operador permite explorar valores intermedios entre soluciones ya evaluadas, favoreciendo la convergencia hacia regiones prometedoras del espacio de búsqueda.

5.4 Mutación

Para mantener diversidad en la población y evitar la convergencia prematura, se aplica un operador de mutación que introduce una pequeña variación aleatoria sobre el valor de K_p del individuo generado. La mutación se implementa añadiendo ruido uniforme [3] en el rango $[-0.5, 0.5]$, con una probabilidad de aplicación del 10%. De esta forma se asegura que el algoritmo pueda explorar nuevas soluciones fuera de las combinaciones lineales de los ascendientes, evitando estancamientos en óptimos locales.

5.5 Implementación

Tanto el algoritmo genético como el lazo de control proporcional se implementa en el lenguaje de programación *Python* siguiendo [8]. El esquema de control está basado en el expuesto en [1].

6 Control Neuronal

El propósito del control neuronal es aprovechar la capacidad de las redes neuronales para aprender comportamientos a partir de datos registrados, sin necesidad de definir explícitamente reglas lingüísticas o parámetros de control. En este proyecto se empleó una red neuronal sencilla entrenada *offline* con los registros obtenidos mediante el controlador borroso y el sensor de giro para predecir la velocidad angular de corrección en porcentaje que es necesario aplicar. Por tanto, se trata de un problema de regresión.

6.1 Dataset

El *dataset* se construyó a partir de las variables medidas por los encoders y el sensor de giro, junto con la salida del controlador borroso. Cada muestra incluye las posiciones y velocidades de las ruedas, el ángulo de orientación del robot, el error de trayectoria, la corrección aplicada y la distancia restante al obstáculo. Un ejemplo de registro se muestra a continuación:

```
pos_izq,pos_der,vel_izq,vel_der,yaw, ...
... error_traj,delta_v,dist_restante
-9,61,0,0,-3,0,0,0.359
-21,48,-248,-272,-3,0,0,0.359
```

6.2 Estructura de la red

La red neuronal diseñada corresponde a un perceptrón multicapa (*MLP*) [19] totalmente conectado. Su arquitectura se compone de tres bloques principales:

- Capa de entrada: recibe un vector de 7 características (`pos_izq`, `pos_der`, `vel_izq`, `vel_der`, `yaw`, `error_traj`, `dist_restante`).
- Capas ocultas:
 - Primera capa oculta con 32 neuronas y función de activación *ReLU*.
 - Segunda capa oculta con 16 neuronas, también con activación *ReLU*.
- Capa de salida: formada por una única neurona lineal que produce el valor de corrección angular Δv (`delta_v`), ya que se trata de un problema de regresión. Esta salida se interpreta como la acción, es decir, la corrección de velocidad angular en porcentaje que el robot debe aplicar para ajustar su trayectoria en tiempo real.

6.3 Entrenamiento

El objetivo del aprendizaje es aprender qué corrección angular Δv debe inferir según la información del entorno que recibe, es decir, las 7 entradas descritas anteriormente. Para ello, aprende la etiqueta `delta_v` del *dataset*. Se ha entrenado mediante descenso de gradiente estocástico (*SGD*) con el optimizador Adam y función de pérdida *MSE* (error cuadrático medio).

Los parámetros del entrenamiento se recogen en la tabla 5.

Tabla 5: Parámetros del entrenamiento de la red neuronal.

Parámetro	Valor
Entradas	Posiciones y velocidades, <i>yaw</i> , error, distancia.
Salida	Corrección vel. angular Δv en porcentaje. Función de activación lineal (regresión).
Capas ocultas	2, con 32 y 16 neuronas respectivamente y función de activación <i>ReLU</i> .
Función de pérdida	<i>MSE</i> .
Optimizador	Adam.
<i>Learning rate</i> y épocas	0.001, 200.

6.4 Implementación

La fase de entrenamiento se ha llevado a cabo mediante el lenguaje de programación *Python* y la librería *PyTorch* [15], mientras que la fase de inferencia se ha programado en el lenguaje *C++* utilizando la librería *LibTorch* [12].

Para obtener el *dataset* se ha añadido al nodo de *ROS2* del controlador borroso un publicador de los datos de interés para que otro nodo los reciba y se encargue de almacenarlos en un fichero *.csv*.

El modelo entrenado se ha exportado a un fichero *.pt* o *TorchScript* [20].

7 Resultados

Los experimentos muestran que los controladores propuestos logran mantener la trayectoria recta del robot en presencia de diferencias entre motores y perturbaciones externas. Los gráficos presentados se han creado con *Matlab* [14].

7.1 Control fuzzy

La figura 5 muestra la evolución de la señal de error para los tres controladores borrosos implementados: el basado en el modelo geométrico de velocidades, el de posiciones de los motores y el que emplea directamente el sensor de giro. Se aprecia claramente cómo la elección de la ganancia influye de manera decisiva en la estabilidad de la respuesta. En particular, el controlador que utiliza las posiciones de los motores presenta una oscilación significativamente mayor, lo que evidencia la necesidad de ajustar cuidadosamente la ganancia para evitar sobrecompensaciones.

En contraste, el controlador con el sensor de giro y una ganancia de 2.5 ofrece la mejor respuesta: apenas oscila y mantiene una trayectoria estable, corrigiendo las desviaciones de forma rápida y eficaz.

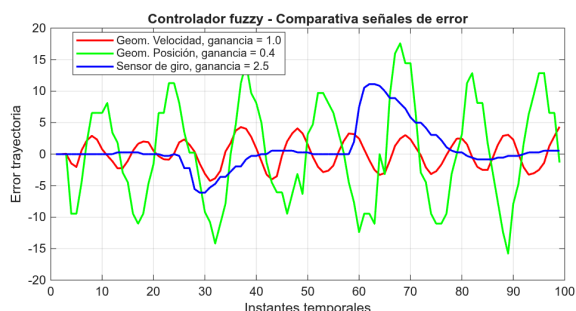


Figura 5: Comparativa de señales de error del controlador *fuzzy*.

7.2 Algoritmos genéticos

La tabla 6 muestra la evolución de la población en distintas generaciones, donde se aprecia cómo los valores de K_p tienden progresivamente hacia regiones más estables y con menor *fitness*. Este comportamiento refleja la capacidad del algoritmo para explorar el espacio de búsqueda y converger hacia parámetros que mejoran la respuesta del controlador. En particular, se observa que valores intermedios de K_p ofrecen un equilibrio adecuado entre rapidez de corrección y ausencia de oscilaciones excesivas, estando en $K_p = 5.31$ el óptimo logrado.

Tabla 6: Evolución del algoritmo genético en las distintas generaciones.

Gen.	Indiv.	K_p	Fitness
0	0	8.1319	10.7280
0	1	6.6784	8.9099
0	2	4.0937	12.3187
0	3	5.4566	11.2959
0	4	5.7875	11.3842
0	5	5.0053	14.5987
1	0	6.6784	12.0384
1	1	8.1319	9.8214
1	2	5.2523	10.1088
1	3	5.3148	7.3046
1	4	5.5053	11.2158
1	5	7.8817	11.3902
2	0	5.3148	6.7274
2	1	8.1319	9.1407
2	2	5.3319	8.6478
2	3	5.4473	7.4172
2	4	5.2687	9.8123
2	5	7.6908	13.5509
3	0	8.1319	10.1570
3	1	5.3148	9.1667
3	2	7.9424	13.0775
3	3	5.2710	11.1285
...			

Si se grafica la señal del error respecto al tiempo, utilizando los valores poblacionales límite 2.0 y 9.0 junto con el valor óptimo obtenido 5.31, figura 6, se observa que este último constituye efectivamente una solución adecuada, ya que proporciona una respuesta relativamente rápida con menor nivel de oscilación. En cambio, la señal correspondiente a $K_p = 2.0$ (curva roja) muestra una corrección demasiado lenta, mientras que la de $K_p = 9.0$ (curva azul) presenta un exceso de oscilaciones, generando picos más pronunciados y elevados que los observados en la señal verde correspondiente a $K_p = 5.31$.

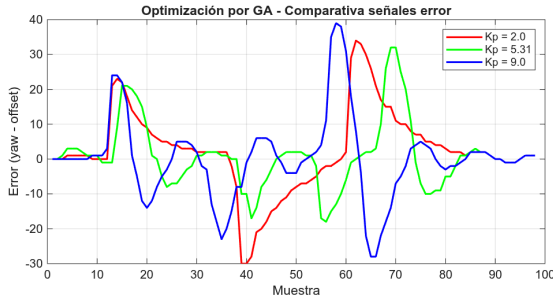


Figura 6: Comparativa de señales de error con K_p optimizado por GA.

7.3 Control neuronal

La figura 7 muestra la evolución de la señal de error para el controlador neuronal. Se observa que una vez estable, el controlador es capaz de contrarrestar las perturbaciones pequeñas, pero cuando se enfrenta a una gran perturbación, su respuesta introduce un giro a la izquierda, aumentando el error, y después, comienza a corregir. Este sesgo se debe a que el controlador neuronal no actúa bajo una ley de control que gobierna la dinámica, sino bajo un comportamiento aprendido.

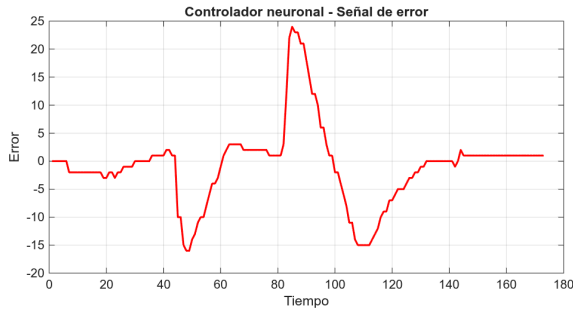


Figura 7: Señal de error del controlador neuronal.

7.4 Controlador borroso VS controlador proporcional tradicional

La figura 5 para el controlador borroso con sensor de giro y la 6 para $K_p = 5.31$ muestran los resultados de los mejores controladores obtenidos. A simple vista, el controlador *fuzzy* parece responder mejor a las perturbaciones, pero no se puede confirmar con dichos gráficos porque el controlador proporcional tradicional está haciendo frente a perturbaciones mayores.

Por tanto, en la figura 8 se presenta la comparativa de ambos sistemas con perturbaciones similares. Las señales de error se han desfasado para comparar con mayor claridad, y aunque el controlador borroso, señal azul, hace frente a perturbaciones algo mayores, muestra la misma estabilidad que el controlador proporcional, siendo este un claro in-

dicador de la robustez del controlador basado en lógica difusa.

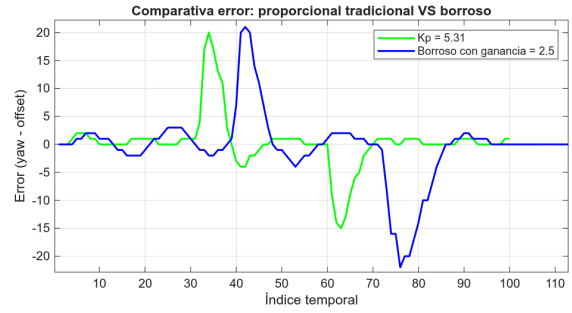


Figura 8: Comparativa de los controladores proporcional tradicional y borroso.

8 Conclusiones

Los resultados obtenidos confirman que la información directa del giroscopio constituye la fuente más fiable para la estimación del error de trayectoria, ya que en caso contrario, el movimiento puede verse afectado por el rozamiento, entre otros, y el modelo geométrico no lo contempla.

Se pone de manifiesto que la ganancia es un parámetro importante en el diseño de controladores borrosos puesto que permiten reajustar externamente la respuesta ofrecida por el controlador, y en este caso, reutilizar el motor de inferencia para diferentes sistemas.

Como se observa en los resultados del algoritmo genético, el ajuste óptimo de los parámetros K o ganancias, depende tanto del modelo empleado como de las perturbaciones presentes en el entorno, siendo necesarios entornos de simulación con perturbaciones homogéneas. También depende fundamentalmente de la función de aptitud definida, siendo esta la componente con mayor transcendencia de los algoritmos genéticos.

Además, se ha demostrado el gran potencial de los controladores *fuzzy* para ser una alternativa a los controladores tradicionales. Si bien se puede pensar que estos sistemas eliminarían la necesidad de ajuste de los parámetros K de los controladores clásicos, la ganancia introducida contrarresta esa posible ventaja.

Respecto al controlador neuronal, se ha detectado un sesgo hacia correcciones excesivas a la izquierda, derivado del comportamiento del *dataset* y del controlador borroso empleado como referencia. Esto se debe a que el controlador neuronal no captura la dinámica del sistema, sino un comportamiento aprendido, lo que limita su capacidad de generalización. Para eliminar estas limitaciones el conjunto de datos de entre-

namiento tendría que generarse bajo condiciones homogéneas, es decir, en simulación, y explorar diferentes técnicas como aprendizaje por refuerzo (*reinforcement learning*).

Finalmente, con los resultados presentados y las perturbaciones no homogéneas aplicadas a los distintos sistemas, no se puede confirmar de manera concluyente que el controlador borroso tenga mejor estabilidad que el proporcional tradicional correctamente ajustado. Para una comparación justa sería necesario someter ambos a las mismas condiciones de perturbación. En cualquier caso, se puede concluir que el controlador borroso constituye una alternativa válida a los controladores clásicos, y que a pesar de someterse a mayores perturbaciones externas, mantiene la misma proporción de estabilidad que el proporcional.

9 Líneas futuras

Dadas las limitaciones académicas bajo las que se ha desarrollado este trabajo, expuestas en el apartado 2, existen numerosas líneas de continuación del proyecto.

En primer lugar, y la más destacada, es la simulación de los algoritmos mediante algún *software*, por ejemplo, Gazebo [6]. Este programa sería muy adecuado puesto que permite reutilizar todo el desarrollo. El *framework ROS2* posibilita utilizar los mismos canales de comunicación para el robot simulado como para el real, por lo que sería suficiente con desconectar el robot real y conectar el virtual. Estas simulaciones permitirían probar, ejecutar y entrenar las técnicas propuestas bajo las mismas condiciones de entorno, principalmente rozamiento y perturbaciones externas, garantizando una comparativa justa entre los distintos métodos.

Además, las simulaciones permitirían extraer un *dataset* sin el sesgo observado en los resultados y en las conclusiones, dando lugar a un controlador neuronal más fiel a la tarea de corrección de trayectoria y ejecutar la optimización genética con mayor velocidad y mayor número de generaciones. También podría entrenarse el controlador neuronal de manera *online* mientras se ejecuta el mejor controlador obtenido.

En segundo lugar, pero relacionado con la mencionada simulación, el diseño de un controlador PID discreto tradicional optimizado mediante el algoritmo genético planteado, que al tener que optimizar tres parámetros resultaba muy tedioso realizarlo con el robot real. Cada individuo pasaría a ser un vector de 3 valores, siendo necesario adaptar las operaciones a este vector, y podría incorporar una componente neuronal que comprobase

que el nuevo individuo es coherente respecto a las reglas clásicas de sintonización y comportamiento de constantes según la dinámica del sistema en cuestión. Por ejemplo, un individuo con baja K_p y el resto de constantes altas, a priori, no tendría mucho sentido y retrasaría la convergencia.

En tercer lugar, la optimización de las ganancias del controlador *fuzzy* mediante el algoritmo genético propuesto.

En cuarto lugar, estudiar y comparar (*benchmark*) los efectos de introducir diferentes pesos a las componentes de la función de aptitud del algoritmo genético.

Por último y en quinto lugar, superar la limitación del controlador neuronal mediante un controlador basado en aprendizaje por refuerzo, con el objetivo de ajustar la velocidad de corrección en función de la distancia restante al objeto. Esta rama de trabajo buscaría que el robot corrija progresivamente la trayectoria para que finalice el movimiento en la orientación y posición teórica si no hubiese sufrido perturbaciones.

English summary

Intelligent control techniques applied to the educational robot Lego Mindstorms EV3

Abstract

This paper presents the design and implementation of different intelligent control techniques applied to a Lego Mindstorms EV3 educational differential robot. Fuzzy controllers, proportional controllers optimized using genetic algorithms, and neural controllers are developed with the aim of correcting deviations in the trajectory without relying exclusively on external sensors or manual tuning of classical controllers. The results validate the viability of these techniques in an educational robotics environment.

Keywords: educational robotics, intelligent control, fuzzy controller, neural controller, optimization, simulation.

Referencias

- [1] Arambarri Calvo, Javier, & Arambarri Calvo, Josu. (2024). *EXPLORANDO LA ROBÓTICA: Guía para First Lego League y World Robot Olympiad*. España: Amazon KDP, octubre de 2024. URL: <https://www.amazon.es/dp/B0DJDM1J8>
- [2] Arambarri Calvo, Javier. (2025). *eduROS2: robótica educativa Lego con ROS2*. Repositorio en GitHub. Disponible en: <https://github.com/arambarricalvoj/eduROS2/tree/ROSConEs25>
- [3] Bäck, T. (1996). *Evolutionary Algorithms in Theory and Practice: Evolution Strategies, Evolutionary Programming, Genetic Algorithms*. Oxford University Press.
- [4] Pedrycz, W. (1993). *Fuzzy sets and fuzzy systems*. Research Studies Press, England.
- [5] Zadeh, L. (1965). "Fuzzy logic". *Fuzzy Sets and Systems*, pp 100-106.
- [6] Open Source Robotics Foundation. (2025). *Gazebo Simulator*. Disponible en: <https://gazebo.org>
- [7] Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Reading, MA: Addison-Wesley.
- [8] Gutiérrez Reina, D., Tapia Córdoba, A., & Rodríguez del Nozal, Á. (2020). *Algoritmos Genéticos con Python: Un enfoque práctico para resolver problemas de ingeniería*. Barcelona: Marcombo.
- [9] Herrera, F., Lozano, M., & Sánchez, A. M. (2004). *Operadores de Cruce con Múltiples Descendientes para Algoritmos Genéticos con Codificación Real: Estudio Experimental*. Congreso Español sobre Metaheurísticas, Algoritmos Evolutivos y Bioinspirados (MAEB).
- [10] Kozielski, M., Prokopowicz, P., & Mikołajewski, D. (2024). Aggregators Used in Fuzzy Control—A Review. *Electronics*, 13(16), 3251. doi:10.3390/electronics13163251. Disponible en: <https://www.mdpi.com/2079-9292/13/16/3251>
- [11] LEGO Education. (2013). *LEGO Mindstorms EV3*. Disponible en: <https://education.lego.com/en-us/product/mindstorms-ev3>
- [12] PyTorch Foundation. (2025). *LibTorch C++ API*. Disponible en: <https://pytorch.org/cppdocs/>
- [13] Macenski, S., Foote, T., Gerkey, B., Lalancette, C., & Woodall, W. (2022). Robot Operating System 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66), eabm6074. doi:10.1126/scirobotics.abm6074. Disponible en: <https://www.science.org/doi/abs/10.1126/scirobotics.abm6074>
- [14] MathWorks. (2025). *MATLAB Online*. Disponible en: <https://www.mathworks.com/products/matlab-online.html>
- [15] PyTorch Foundation. (2025). *PyTorch Documentation*. Disponible en: <https://pytorch.org/docs/stable/index.html>
- [16] Rada-Vilela, J. (2016). *FuzzyLite: A Fuzzy Logic Control Library in C++*. Disponible en: <https://fuzzylite.com>
- [17] Ross, T. J. (2010). *Fuzzy Logic with Engineering Applications*. Wiley, 3rd Edition.
- [18] Siegart, R., Nourbakhsh, I., & Scaramuzza, D. (2011). *Introduction to Autonomous Mobile Robots*. MIT Press.
- [19] Stevens, E., Antiga, L., & Viehmann, T. (2020). *Deep Learning with PyTorch*. Sebastopol, CA: O'Reilly Media.
- [20] PyTorch Foundation. (2025). *TorchScript*. Disponible en: <https://pytorch.org/docs/stable/jit.html>



© 2025 por el autor bajo licencia CC BY-NC-SA 4.0 (<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.es>).